

Architectural Synthesis

Topics: Circuit Specification, Architectural Optimization, Data-Path Synthesis, Control Unit Synthesis, Pipelined Circuits, ASIC, FPGA and PLDs

1. What is Architectural Synthesis?

- Architectural Synthesis converts a circuit specification into a hardware architecture.
- Specification $\rightarrow$ Datapath + Control Unit $\rightarrow$ Hardware implementation
- Example: $Y = (A + B) \times C$
- Required hardware may include an adder, multiplier, registers, multiplexers and control signals.

2. Circuit Specification

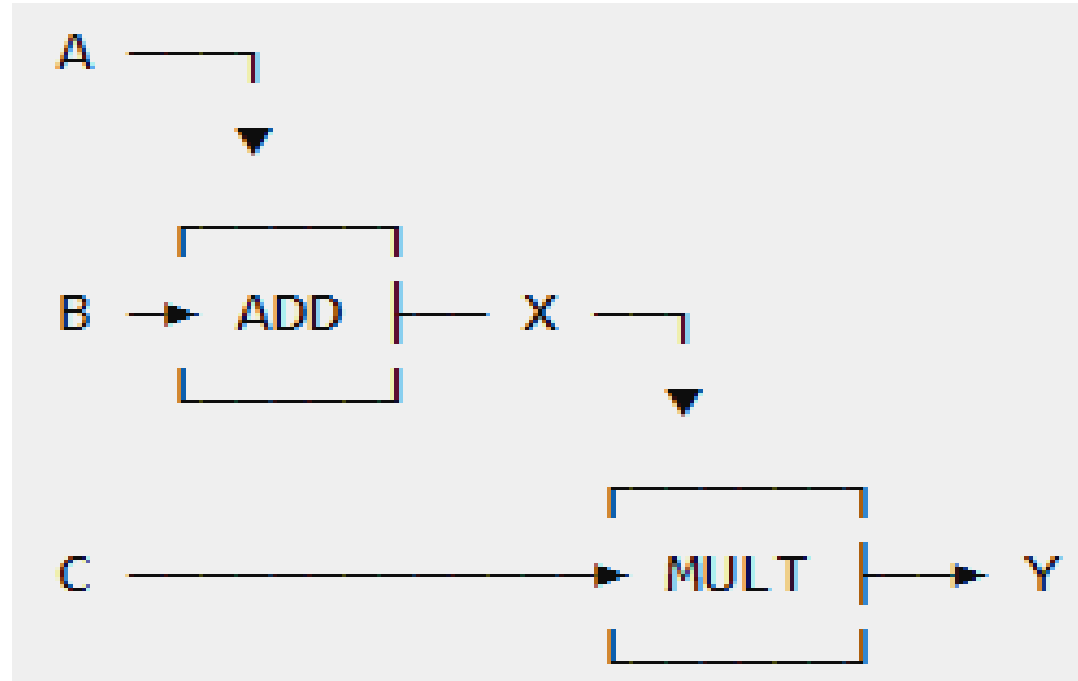
- Circuit specification describes **WHAT** the circuit should **do**.
- Example: Two **8-bit** inputs **A** and **B** are multiplied to **produce a 16-bit output Y**.
- $A = 8\text{-bit}, B = 8\text{-bit}, Y = A \times B$
- At this stage, we define **function, inputs, outputs and constraints**—not the detailed hardware.

3. Architectural Optimization

- Goal: implement the same function with **better hardware efficiency**.
- Common objectives:
 - reduce area,
 - increase speed,
 - reduce power and
 - reduce cost.
- Key trade-off: Area $\leftrightarrow$ Speed $\leftrightarrow$ Power
- Example: Hardware **resources may be shared when operations occur in different clock cycles**.

4. Data-Path Synthesis

- Datapath is the part of the circuit that stores, moves and processes data.
- Typical components:
 - Registers,
 - ALU,
 - Adders,
 - Multipliers,
 - Comparators,
 - Multiplexers and
 - Shifters.
- Example: $Y = (A + B) \times C$
- The datapath performs ADD first, then MULTIPLY, and finally stores the result.



6. Control Unit Synthesis

The Control Unit determines **WHEN** each data-path operation occurs.

Example sequence:

$$Y = (A + B) \times C$$

Control Unit বলে দেয় কখন কোন hardware কাজ করবে।

S0: Load A, B, C

S1: $A + B \rightarrow R1$

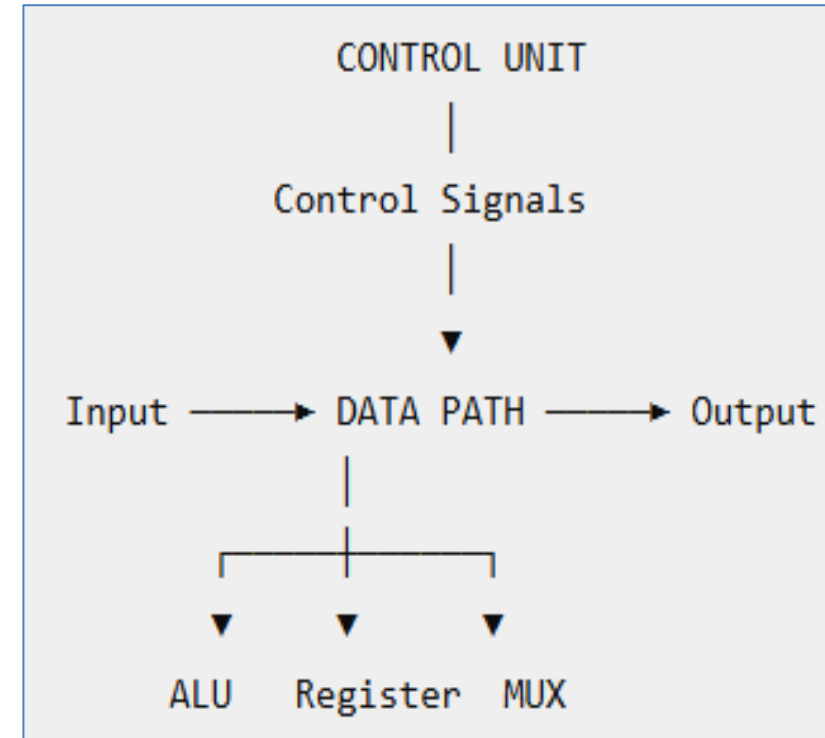
S2: $R1 \times C \rightarrow R2$

S3: $R2 \rightarrow Y$

Typical control signals: LOAD, ADD, MULT, STORE.

7. Datapath + Control Unit

- Datapath = performs the operations.
- Control Unit = generates signals that control the operations.
- Control Unit → Control Signals → Datapath
- Together they form the architecture of a digital system.



Datapath = কাজ করে

Control Unit = কাজ করায়

8. Pipelined Circuits

- Pipelining divides a computation into multiple stages.
- Different data items can occupy different stages at the same time.
- Example: Stage 1 → Stage 2 → Stage 3 → Stage 4
- Main benefit: higher throughput.
- Important: Pipeline can improve throughput without necessarily reducing the latency of one data item.

Pipeline Math — Easy Explanation

Suppose a circuit has **4 stages**:

</> Plain text

Stage 1 → Stage 2 → Stage 3 → Stage 4

Their processing times are:

- Stage 1 = 20 ns
- Stage 2 = 30 ns
- Stage 3 = 25 ns
- Stage 4 = 15 ns

1. Without Pipelining

For **one data item**, all 4 stages must be completed one after another.

So,

$$T = 20 + 30 + 25 + 15$$

$$T = 90ns$$

Therefore,

1 data item takes 90 ns.

Suppose we have **4 data items**: D1, D2, D3 and D4.

</> Plain text

D1 → 90 ns

D2 → 90 ns

D3 → 90 ns

D4 → 90 ns

Therefore,

$$T_{non-pipeline} = 4 \times 90$$

$$\boxed{360ns}$$

2. Now Use Pipelining

We divide the circuit into 4 pipeline stages:

</> Plain text

Stage 1	Stage 2	Stage 3	Stage 4
20 ns	30 ns	25 ns	15 ns

The **slowest stage** takes 30 ns.

Therefore,

$$T_{clock} = \max(20, 30, 25, 15)$$

$$T_{clock} = 30ns$$

Why 30 ns?

Because the clock must be long enough for **every stage** to complete its work.

3. Processing 4 Data Items

Now we have:

</> Plain text

D1, D2, D3, D4

The pipeline works like this:

Clock Cycle	Stage 1	Stage 2	Stage 3	Stage 4
1	D1			
2	D2	D1		
3	D3	D2	D1	
4	D4	D3	D2	D1
5		D4	D3	D2
6			D4	D3
7				D4

Notice something important:

D1 is being processed in Stage 2 while D2 is already entering Stage 1.

That is the main idea of **pipelining**.

3. Processing 4 Data Items

Now we have:

</> Plain text

D1, D2, D3, D4

The pipeline works like this:

Clock Cycle	Stage 1	Stage 2	Stage 3	Stage 4
1	D1			
2	D2	D1		
3	D3	D2	D1	
4	D4	D3	D2	D1
5		D4	D3	D2
6			D4	D3
7				D4

Notice something important:

D1 is being processed in Stage 2 while D2 is already entering Stage 1.

That is the main idea of **pipelining**.

4. Pipeline Time

There are:

$$k = 4$$

stages.

And:

$$n = 4$$

data items.

The formula is:

$$T_{\text{pipeline}} = (k + n - 1)T_{\text{clk}}$$

Substitute the values:

$$T_{\text{pipeline}} = (4 + 4 - 1) \times 30$$

$$= 7 \times 30$$

$$\boxed{210ns}$$

So, 4 data items take 210 ns with pipelining.

4. Pipeline Time

There are:

$$k = 4$$

stages.

And:

$$n = 4$$

data items.

The formula is:

$$T_{pipeline} = (k + n - 1)T_{clock}$$

Substitute the values:

$$\begin{aligned} T_{pipeline} &= (4 + 4 - 1) \times 30 \\ &= 7 \times 30 \\ &= \boxed{210ns} \end{aligned}$$

So, 4 data items take 210 ns with pipelining.

5. Compare the Two

Without Pipeline

$$T = 360ns$$

With Pipeline

$$T = 210ns$$

Therefore,

$$360 > 210$$

So, pipelining saves time when processing multiple data items.

6. Calculate Speedup

Speedup tells us how much faster the pipelined system is.

$$Speedup = \frac{T_{Non-pipeline}}{T_{Pipeline}}$$

Therefore,

$$Speedup = \frac{360}{210}$$

$$\boxed{Speedup \approx 1.71}$$

So, the pipelined circuit is approximately 1.71 times faster for these 4 data items.

5. Compare the Two

Without Pipeline

$$T = 360ns$$

With Pipeline

$$T = 210ns$$

Therefore,

$$360 > 210$$

So, pipelining saves time when processing multiple data items.

6. Calculate Speedup

Speedup tells us how much faster the pipelined system is.

$$Speedup = \frac{T_{Non-pipeline}}{T_{Pipeline}}$$

Therefore,

$$Speedup = \frac{360}{210}$$

$Speedup \approx 1.71$

So, the pipelined circuit is approximately **1.71 times faster** for these 4 data items.

7. Very Simple Example

Think about a car washing system with 4 steps:

</> Plain text

Wash → Clean → Dry → Polish

Without pipelining:

</> Plain text

Car 1: Wash → Clean → Dry → Polish

Car 2: Wash → Clean → Dry → Polish

Car 3: Wash → Clean → Dry → Polish

One car must finish completely before the next car starts.

With pipelining:

</> Plain text

	Wash	Clean	Dry	Polish	
Car 1	✓	✓	✓	✓	
Car 2		✓	✓	✓	✓
Car 3			✓	✓	✓

Different cars can be at different stages **at the same time**.

That's exactly how a pipelined processor/circuit works.

★ Important Formulas for Exam

1. Non-pipelined time for one data item

$$T_{non-pipeline} = T_1 + T_2 + \dots + T_k$$

2. Pipeline clock period

$$T_{clock} = \max(T_1, T_2, \dots, T_k)$$

3. Pipeline time for n data items

$$T_{pipeline} = (k + n - 1)T_{clock}$$

4. Speedup

$$Speedup = \frac{T_{Non-pipeline}}{T_{Pipeline}}$$

Pipelining increases throughput by allowing multiple data items to be processed simultaneously in different stages.

10. ASIC, FPGA and PLD

- ASIC: Application-Specific Integrated Circuit; designed for a specific application.
- FPGA: Field-Programmable Gate Array; configurable after manufacturing.
- PLD: Programmable Logic Device; programmable digital logic hardware.
- ASIC: high performance and low power, but less flexible.
- FPGA: highly flexible and suitable for prototyping and changing designs.

ASIC

ASIC = Application-Specific Integrated Circuit

একটি নির্দিষ্ট application-এর জন্য বিশেষভাবে তৈরি chip।

Example:

- Mobile processor-এর নির্দিষ্ট hardware
- Network processing chip
- Cryptocurrency accelerator

ASIC সাধারণত:

- High performance
- Low power
- High volume production-এর জন্য ভালো

কিন্তু design ও manufacturing cost বেশি।



10. ASIC, FPGA and PLD

- ASIC: Application-Specific Integrated Circuit; designed for a specific application.
- FPGA: Field-Programmable Gate Array; configurable after manufacturing.
- PLD: Programmable Logic Device; programmable digital logic hardware.
- ASIC: high performance and low power, but less flexible.
- FPGA: highly flexible and suitable for prototyping and changing designs.

- FPGA হলো এমন একটি programmable chip, যেটাকে ব্যবহার করে আমরা প্রয়োজন অনুযায়ী বিভিন্ন ধরনের digital circuit তৈরি করতে পারি।
- অর্থাৎ একই FPGA-কে আজ Counter, কাল ALU, আবার অন্যদিন Processor হিসেবে configure করা যায়।

10. ASIC, FPGA and PLD

- ASIC: Application-Specific Integrated Circuit; designed for a specific application.
- FPGA: Field-Programmable Gate Array; configurable after manufacturing.
- PLD: Programmable Logic Device; programmable digital logic hardware.
- ASIC: high performance and low power, but less flexible.
- FPGA: highly flexible and suitable for prototyping and changing designs.

একটা বাস্তব উদাহরণ

ধরুন আপনার কাছে একটি খালি ঘর আছে।

ঘরটি এখনো কোনো নির্দিষ্ট কাজে ব্যবহার করা হয়নি।

আজ আপনি ঘরটিকে বানালেন:

</> Plain text

Classroom

কিছুদিন পর furniture পরিবর্তন করে বানালেন:

</> Plain text

Office

আবার পরে বানালেন:

</> Plain text

Laboratory

ঘরটি একই, কিন্তু ভেতরের arrangement পরিবর্তন করে কাজ পরিবর্তন করা যাচ্ছে।

FPGA-ও অনেকটা এমন।

10. ASIC, FPGA and PLD

- ASIC: Application-Specific Integrated Circuit; designed for a specific application.
- FPGA: Field-Programmable Gate Array; configurable after manufacturing.
- PLD: Programmable Logic Device; programmable digital logic hardware.
- ASIC: high performance and low power, but less flexible.
- FPGA: highly flexible and suitable for prototyping and changing designs.

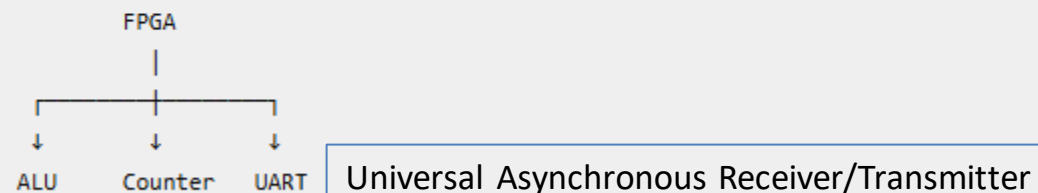
FPGA কীভাবে কাজ করে?

FPGA-এর ভিতরে অনেকগুলো programmable logic resources থাকে।

আমরা configuration/programming-এর মাধ্যমে ঠিক করি এগুলো কীভাবে connected থাকবে এবং কী logic করবে।

যেমন:

</> Plain text



একই FPGA-তে প্রয়োজন অনুযায়ী বিভিন্ন circuit implement করা যায়।

10. ASIC, FPGA and PLD

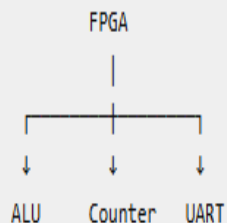
FPGA কীভাবে কাজ করে?

FPGA-এর ভিতরে অনেকগুলো programmable logic resources থাকে।

আমরা configuration/programming-এর মাধ্যমে ঠিক করি এগুলো কীভাবে connected থাকবে এবং কী logic করবে।

যেমন:

</> Plain text



Universal Asynchronous Receiver/Transmitter

একই FPGA-তে প্রয়োজন অনুযায়ী বিভিন্ন circuit implement করা যায়।

একটি সহজ Example

ধরুন আমরা FPGA-তে একটি Counter তৈরি করলাম।

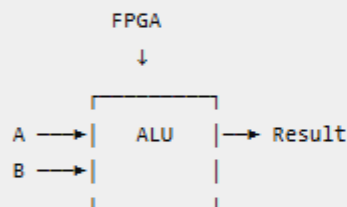
</> Plain text



0 → 1 → 2 → 3 → 4 → 5 → ...

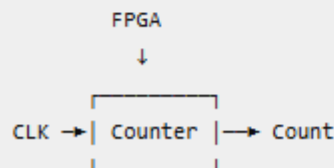
পরে যদি Counter-এর দরকার না হয়, FPGA-কে নতুনভাবে configure করে সেখানে একটি ALU তৈরি করা যেতে পারে।

</> Plain text



আবার অন্য design configure করলে:

</> Plain text



Chip একই, কিন্তু implemented hardware logic পরিবর্তন করা যায়।

10. ASIC, FPGA and PLD

FPGA বনাম সাধারণ Chip

ধরুন একটি সাধারণ Calculator IC শুধু নির্দিষ্ট কাজ করে।

</> Plain text

Chip

↓

Calculator function

কিন্তু FPGA:

</> Plain text

FPGA

├ Counter

├ ALU

├ UART

├ Processor

└ Digital Filter

প্রয়োজন অনুযায়ী FPGA-কে নতুনভাবে configure করা যায়।

FPGA কেন ব্যবহার করি?

1. Design পরিবর্তন করা যায়

Hardware prototype তৈরি করার সময় খুব useful।

2. দ্রুত পরীক্ষা করা যায়

নতুন circuit design করে FPGA-তে implement করে test করা যায়।

3. একই FPGA-তে বিভিন্ন circuit তৈরি করা যায়

আজ Counter, কাল ALU—প্রয়োজন অনুযায়ী।

4. ASIC-এর আগে prototype হিসেবে ব্যবহার করা যায়

অনেক সময় final ASIC তৈরি করার আগে FPGA-তে design পরীক্ষা করা হয়।

ASIC:

একবার তৈরি করলে নির্দিষ্ট কাজের জন্য তৈরি chip।

FPGA:

তৈরি হওয়ার পরও configuration-এর মাধ্যমে hardware logic পরিবর্তন করা যায়।

10. ASIC, FPGA and PLD

PLD কী?

PLD = Programmable Logic Device

সহজ কথায়:

PLD হলো এমন একটি electronic chip, যেটাকে আমরা নিজের প্রয়োজন অনুযায়ী digital logic circuit হিসেবে configure করতে পারি।

অর্থাৎ, chip-টি আগে থেকেই শুধু একটি নির্দিষ্ট কাজের জন্য fixed নয়।

PLD-এর বিভিন্ন ধরন

সহজভাবে:

</> Plain text



1. SPLD

Simple Programmable Logic Device

ছোট এবং relatively simple logic circuit-এর জন্য।

2. CPLD

Complex Programmable Logic Device

SPLD-এর তুলনায় বেশি complex logic implement করতে পারে।

3. FPGA

Field Programmable Gate Array

অনেক বেশি logic এবং complex digital system implement করতে পারে।

10. ASIC, FPGA and PLD

PLD-এর বিভিন্ন ধরন

সহজভাবে:

</> Plain text



1. SPLD

Simple Programmable Logic Device

ছোট এবং relatively simple logic circuit-এর জন্য।

2. CPLD

Complex Programmable Logic Device

SPLD-এর তুলনায় বেশি complex logic implement করতে পারে।

3. FPGA

Field Programmable Gate Array

অনেক বেশি logic এবং complex digital system implement করতে পারে।

তাহলে FPGA কি PLD?

হ্যাঁ।

FPGA হলো PLD-এর একটি advanced/large type।

এভাবে মনে রাখুন:

</> Plain text

PLD

|

├ Simple → SPLD

|

├ More Complex → CPLD

|

└ Very Large/Complex → FPGA

🏠 সহজ বাস্তব উদাহরণ

একটি বাড়ির কথা ভাবুন।

SPLD = ছোট একটি room

CPLD = একটি ছোট বাড়ি

FPGA = বড় একটি বাড়ি/complex

তিনটিই programmable, কিন্তু capacity ও complexity আলাদা।

11. ASIC vs FPGA vs PLD

ASIC — Best for high-volume, finalized products.

FPGA — Best for prototypes, low/medium volume and designs that may change.

PLD — Suitable for programmable logic applications of varying complexity.

ASIC generally offers better performance/power efficiency; FPGA offers greater flexibility.

12. ASIC vs FPGA vs PLD

বিষয়	ASIC	FPGA	PLD
Programmable after manufacturing	না	হ্যাঁ	হ্যাঁ
Speed	খুব বেশি	ভালো	মাঝারি
Flexibility	কম	বেশি	বেশি
Initial cost	বেশি	কম	কম
Production volume	বেশি	কম/মাঝারি	কম
Development	কঠিন	তুলনামূলক সহজ	সহজ